



**Instytut Informatyki
Uniwersytet Rzeszowski**

**Przedmiot:
Programowanie Obiektowe 2**

Esetti

System Zarządzania Kołami Naukowymi i Generowania Dokumentacji

Wykonał:
Kacper Ręczak, 134968

Prowadzący: Mgr inż. Wojciech Gałka

Rzeszów 2026

Spis treści

1	Wstęp	2
2	Architektura Systemu i Użyte Technologie	2
2.1	Uzasadnienie Wyboru Technologii	2
2.2	Zastosowane Wzorce Projektowe	3
2.3	Schemat Warstwowy Aplikacji	3
3	Model Biznesowy i Skalowalność Międzyuczelniana	4
3.1	Opis Schematu Relacyjnego Bazy Danych	4
3.1.1	Tabele Struktur Akademickich	5
3.1.2	Tabele Kont Użytkowników i Uprawnień	5
3.1.3	Tabele Realizacji Projektów i Wydarzeń	6
3.1.4	Tabele Relacji Wiele-do-Wielu (Łączące)	7
4	Moduły Aplikacji i Interfejs Użytkownika	9
4.1	Ekran Startowy (Dashboard)	9
4.2	Nawigacja Systemowa	10
4.3	Moduł Członków Kół (Pełny cykl CRUD)	10
4.3.1	Odczyt danych (Read)	10
4.3.2	Tworzenie rekordu (Create)	11
4.3.3	Modyfikacja rekordu (Update)	12
4.4	Ewidencja Projektów (Pełny cykl CRUD)	13
4.4.1	Odczyt danych i tworzenie (Read & Create)	13
4.5	Rejestr Aktywności i Spotkań (Pełny cykl CRUD)	15
4.5.1	Odczyt danych i rejestracja (Read & Create)	15
4.6	Informacje o Kole Naukowym (Opiekun naukowy)	17
4.7	Generowanie Dokumentów i Raportów	19
4.8	Panel Ustawień i Zarządzania Bazą Danych (Kopia zapasowa i Import)	20
5	Instrukcja Uruchomienia Aplikacji i Ograniczenia	21
5.1	Wymagania Systemowe	21
5.2	Pierwsze Uruchomienie	21
5.3	Znane Ograniczenia Systemowe	22
6	Weryfikacja Systemu (Instrukcja Testowa)	22
6.1	Procedura Testowa: Cykl CRUD Członka Koła	22
6.2	Procedura Testowa: Generowanie Dokumentów PDF	23

1 Wstęp

Projekt Esetti to innowacyjny system desktopowy przeznaczony do kompleksowego wspomaganie działalności kół naukowych, którego architektura została zorientowana na pełną skalowalność międzyuczelnianą. Struktura powiązań bazy danych pozwala na jednoczesną, niezależną obsługę wielu uczelni wyższych oraz ich wydziałów, ułatwiając przy tym ewidencjonowanie wspólnych, wieloosrodkowych projektów badawczych i wyjazdów naukowych bez konieczności zmiany schematu danych.

Kluczowym elementem funkcjonalnym systemu jest moduł automatycznego generowania dokumentów i zaświadczeń w formacie PDF przy użyciu wydajnego silnika QuestPDF. Narzędzie to umożliwia natychmiastowe generowanie szczegółowych list członków koła wraz z pełną obsadą zarządu oraz klauzulami RODO, a także spersonalizowanych certyfikatów poświadczających aktywność studentów w projektach naukowych bezpośrednio na dysk lokalny użytkownika.

2 Architektura Systemu i Użyte Technologie

Aplikacja została zrealizowana w architekturze trójwarstwowej z wyraźnym wydzieleniem odpowiedzialności.

2.1 Uzasadnienie Wyboru Technologii

- **C# i .NET 10.0:** Zapewnia wysoką wydajność, bezpieczeństwo typów oraz dostęp do potężnego ekosystemu bibliotek.
- **Avalonia UI:** Wybrana zamiast technologii takich jak WPF czy WinForms ze względu na jej wieloplatformowość. Umożliwia budowanie aplikacji o nowoczesnym, natywnym wyglądzie dla różnych systemów operacyjnych.
- **SQLite:** Lekka, bezserwerowa relacyjna baza danych. Idealna dla samodzielnej aplikacji desktopowej, gdyż uwalnia końcowego użytkownika z konieczności instalacji zewnętrznego serwera bazodanowego, oferując pełne wsparcie języka SQL.
- **Entity Framework Core:** Zaawansowany ORM pozwalający na mapowanie obiektowo-relacyjne (podejście Code-First). Upraszcza migrację bazy w razie zmiany docelowego silnika (np. na PostgreSQL).
- **QuestPDF:** Narzędzie stosowane do dynamicznego składania sprawozdań o działalności w formacie PDF bezpośrednio z poziomu kodu (Fluent API), z pominięciem niewydajnych generatorów opartych na HTML.

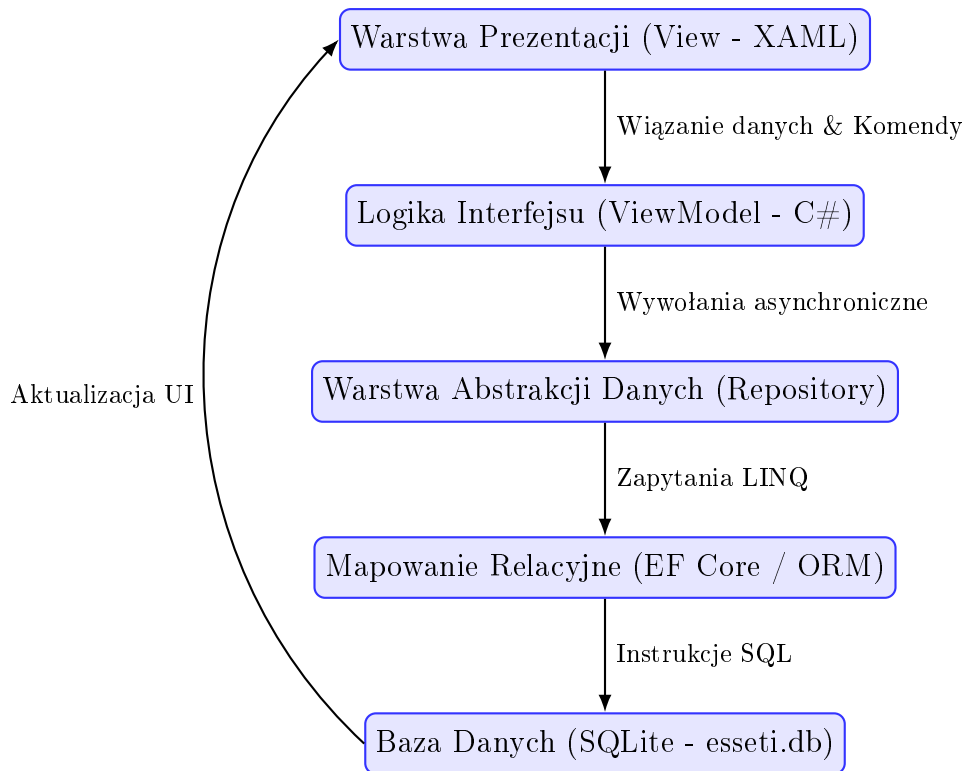
2.2 Zastosowane Wzorce Projektowe

Projekt ściśle trzyma się dobrych praktyk inżynierii oprogramowania (OOP), realizując zaawansowane wzorce:

- **MVVM (Model-View-ViewModel):** Wzorec architektoniczny gwarantujący ścisłą separację logiki interfejsu użytkownika (View) od logiki biznesowej i danych (Model). Komunikacja odbywa się bezstanowo z wykorzystaniem mechanizmów Data Binding i RelayCommand udostępnianych przez pakiet *CommunityToolkit.Mvvm*. Widoki nie posiadają logiki w klasach typu Code-Behind.
- **Wstrzykiwanie Zależności (Dependency Injection / IoC):** Aplikacja wdraża zasadę odwrócenia sterowania (Inversion of Control). Wykorzystano wbudowany kontener *Microsoft.Extensions.DependencyInjection*. Zależności (np. interfejsy repozytoriów, nawigacja) wstrzykiwane są bezpośrednio przez konstruktory.
- **Wzorec Repozytorium (Repository Pattern):** Zaprojektowano warstwę pośrednią pomiędzy logiką biznesową (ViewModelami) a frameworkiem ORM (EF Core). Dedykowane klasy w przestrzeni nazw *Repositories* (np. *ClubRepository*, *MemberRepository*) enkapsulują złożone operacje odczytu i zapisu danych. Zwiększa to testowalność aplikacji i ułatwia ewentualną przyszłą integrację systemu z zewnętrznym interfejsem API.

2.3 Schemat Warstwowy Aplikacji

Poniższy diagram przedstawia przepływ danych i separację warstw w systemie Esetti, od interfejsu graficznego po fizyczny plik bazy danych SQLite.



Rysunek 1: Architektura warstwowa i przepływ danych w aplikacji Esetti

3 Model Biznesowy i Skalowalność Międzyuczelniana

Warstwa danych została ustrukturyzowana tak, aby umożliwić elastyczne skalowanie aplikacji:

- **Hierarchia Akademicka:** Struktura bazuje na encjach `College` (uczelnia), `CollegeDepartment` (wydział) oraz `ClubInfo` (koło naukowe). Pozwala to na jednoczesną obsługę wielu niezależnych organizacji w obrębie jednej bazy danych.
- **Projekty Międzyuczelniane:** Encje `Project` oraz `Trip` (wyjazd naukowy) są połączone z kołami relacjami wiele-do-wielu. Dzięki temu projekt badawczy może być współdzielony przez kilka kół z różnych wydziałów lub uczelni partnerskich.
- **Zarządzanie Tożsamością:** Użytkownik posiada jedno konto systemowe (`UserAccount`), lecz w ramach różnych kół naukowych może pełnić odmienne role (np. przewodniczący sekcji lub zwykły członek), co odzwierciedla tabela pośrednicząca `MemberClub`.

3.1 Opis Schematu Relacyjnego Bazy Danych

Baza danych składa się z 18 tabel powiązanych odpowiednimi relacjami. Poniżej znajduje się szczegółowy opis struktury poszczególnych tabel oraz ich pól:

3.1.1 Tabele Struktur Akademickich

- **college (Uczelnia):**

- `college_id` (PK, int) – klucz główny.
- `name` (string) – pełna nazwa uczelni.
- `name_short` (string) – skrót nazwy uczelni.
- `college_avatar` (blob) – plik graficzny/logo.
- `address_line`, `city`, `postal_code` (string) – dane adresowe.
- `phone` (string) – telefon kontaktowy.
- `NIP` (string) – numer identyfikacji podatkowej.

- **college_department (Wydział Uczelni):**

- `college_department_id` (PK, int) – klucz główny.
- `college_id` (FK, int) – powiązanie z uczelnią.
- `name` (string) – nazwa wydziału.
- `address_line`, `city`, `postal_code`, `phone`, `email` (string) – dane adresowe i teleadresowe wydziału.

- **club_info (Koło Naukowe):**

- `club_id` (PK, int) – klucz główny.
- `name` (string) – nazwa koła.
- `department_id` (FK, int) – powiązanie z wydziałem.
- `club_room` (string) – przydzielona sala spotkań.
- `supervisor_name`, `supervisor_email`, `supervisor_phone` (string) – dane opiekuna koła.
- `meetings_schedule` (string) – opis harmonogramu spotkań.
- `short_name` (string) – nazwa skrócona koła.
- `club_photo` (blob) – logo koła.

3.1.2 Tabele Kont Użytkowników i Uprawnień

- **user_account (Konto Użytkownika):**

- `account_id` (PK, int) – klucz główny.
- `email` (string) – unikalny adres e-mail logowania.

- `password_hash` (string) – skrót hasła logowania.
- `system_role` (string) – rola systemowa.
- `is_verified` (bool) – status weryfikacji.
- `created_at`, `last_login`, `updated_at` (datetime) – metadane konta.

- **member (Członek Koła):**

- `member_id` (PK, int) – klucz główny.
- `account_id` (FK, int) – powiązanie z kontem logowania.
- `role_id` (FK, int) – powiązanie z rolą w kole.
- `index_number` (string) – numer albumu studenta.
- `first_name`, `last_name` (string) – imię i nazwisko.
- `major` (string) – kierunek studiów.
- `phone_number` (string) – telefon kontaktowy.
- `member_avatar` (blob) – zdjęcie profilowe.
- `description` (string) – notatki/opis członka.
- `is_active` (bool) – status aktywności.
- `join_date` (datetime) – data przystąpienia do koła.

- **authority_role (Słownik Uprawnień):**

- `role_id` (PK, int) – klucz główny.
- `name` (string) – nazwa roli (np. Prezes, Opiekun).
- `description` (string) – opis roli.
- `permissions` (string) – zestaw zakodowanych uprawnień.

3.1.3 Tabele Realizacji Projektów i Wydarzeń

- **project (Projekt Naukowy):**

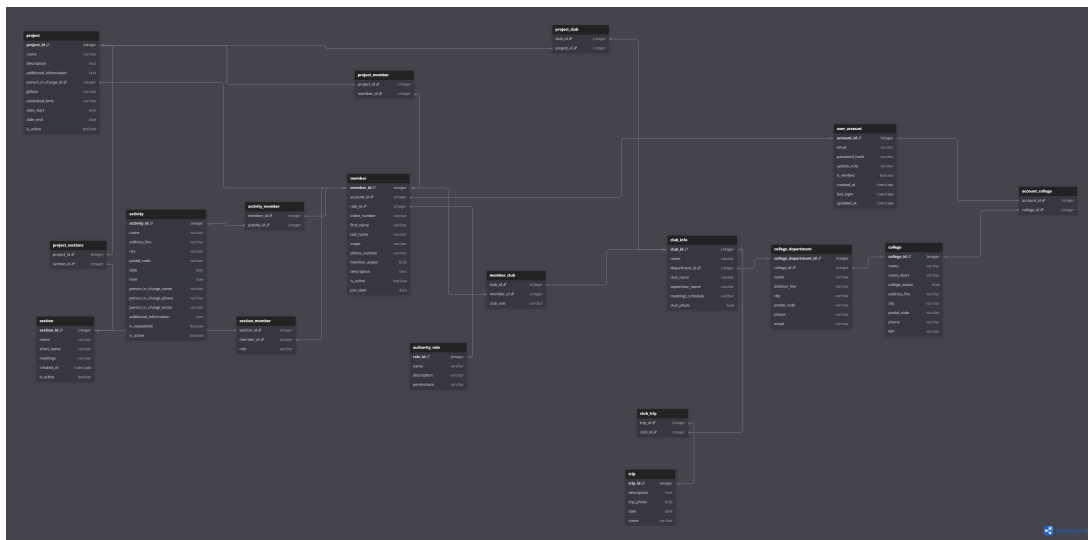
- `project_id` (PK, int) – klucz główny.
- `name` (string) – nazwa projektu.
- `description`, `additional_information` (string) – szczegóły projektu.
- `person_in_charge_id` (FK, int) – kierownik projektu.
- `github` (string) – link do repozytorium kodu.
- `estimated_time` (int) – szacowany czas realizacji w godzinach.

- `date_start`, `date_end` (datetime) – przedział czasowy.
- `is_active` (bool) – status realizacji projektu.
- **section (Sekcja Tematyczna Koła):**
 - `section_id` (PK, int) – klucz główny.
 - `name` (string) – nazwa sekcji.
 - `short_name` (string) – krótka nazwa.
 - `meetings` (string) – harmonogram zebrań sekcji.
 - `created_at` (datetime) – data utworzenia.
 - `is_active` (bool) – status działalności.
- **activity (Wydarzenie/Seminarium):**
 - `activity_id` (PK, int) – klucz główny.
 - `name` (string) – nazwa wydarzenia.
 - `address_line`, `city`, `postal_code` (string) – lokalizacja.
 - `date`, `time` – czas przeprowadzenia wydarzenia.
 - `person_in_charge_name`, `person_in_charge_phone`, `person_in_charge_email` (string) – koordynator.
 - `additional_information` (string) – uwagi organizacyjne.
 - `is_repeatable` (bool) – status cykliczności.
 - `is_active` (bool) – status aktywności wydarzenia.
- **trip (Wyjazd Naukowy):**
 - `trip_id` (PK, int) – klucz główny.
 - `description` (string) – opis wyjazdu naukowego.
 - `trip_photo` (blob) – fotografia z wyjazdu.
 - `date` (datetime) – termin wyjazdu.
 - `name` (string) – nazwa/cel podróży naukowej.

3.1.4 Tabele Relacji Wiele-do-Wielu (Łączące)

- `account_college` – powiązanie kont użytkowników z konkretnymi uczelniami.
- `member_club` – powiązanie członków z kołami wraz z przypisaną rolą w kole (`club_role`).

- `section_member` – powiązanie członków z sekcjami wraz z pełnioną w niej rolą (`role`).
- `project_club` – powiązanie projektów z wieloma kołami naukowymi jednocześnie (umożliwia projekty międzyuczelniane).
- `project_member` – lista członków uczestniczących w projekcie.
- `project_sections` – lista sekcji zaangażowanych w projekt.
- `activity_member` – lista członków zaangażowanych w dane wydarzenie.
- `club_trip` – powiązanie wyjazdów naukowych z kołami.



Rysunek 2: Schemat mapowania obiektowo-relacyjnego (ORM)

4 Moduły Aplikacji i Interfejs Użytkownika

W poniższym rozdziale szczegółowo zaprezentowano strukturę widoków aplikacji, opisano zaawansowaną logikę biznesową stojącą za poszczególnymi modułami oraz zilustrowano pełny cykl życia danych (operacje CRUD – Create, Read, Update, Delete) w systemie Esetti. Każdy moduł został zrealizowany w oparciu o wzorzec projektowy MVVM (Model-View-ViewModel), oddzielający deklaratywną definicję interfejsu (pliki XAML) od kodu sterującego stanem widoku (klasy ViewModel w C#).

4.1 Ekran Startowy (Dashboard)

Pulpit nawigacyjny stanowi centralny punkt aplikacji, wyświetlany bezpośrednio po jej pomyślnym uruchomieniu. Odpowiada za agregowanie i prezentację najważniejszych statystyk oraz powiadomień z różnych modułów w celu natychmiastowego zaprezentowania globalnego stanu koła naukowego.

Klasa `StartViewModel` pobiera dane za pośrednictwem serwisów wstrzykiwanych przez kontener IoC (Inversion of Control). Na poziomie widoku statystyki wyświetlane są w formie nowoczesnych, animowanych kafelków (kart statystycznych) z zaokrąglonymi rogami, subtelnymi cieniami oraz harmonijnie dobranymi przejściami kolorystycznymi w odcieniach ciemnego granatu i błękitu, co nadaje interfejsowi elegancki wygląd klasy Premium. W prawej części pulpitu znajduje się sekcja szybkich skrótów, która umożliwia użytkownikowi natychmiastowe przejście do często używanych funkcji, takich jak dodawanie członka koła czy eksport raportu rocznego.



Rysunek 3: Ekran startowy aplikacji (Dashboard)

4.2 Nawigacja Systemowa

Główne okno aplikacji (*MainWindow*) wdraża ergonomiczny i responsywny boczny panel nawigacyjny oparty na bibliotece Avalonia UI. Nawigacja w systemie Esetti opiera się na autorskim serwisie nawigacyjnym (*NavigationService*), który zarządza stosiem widoków.

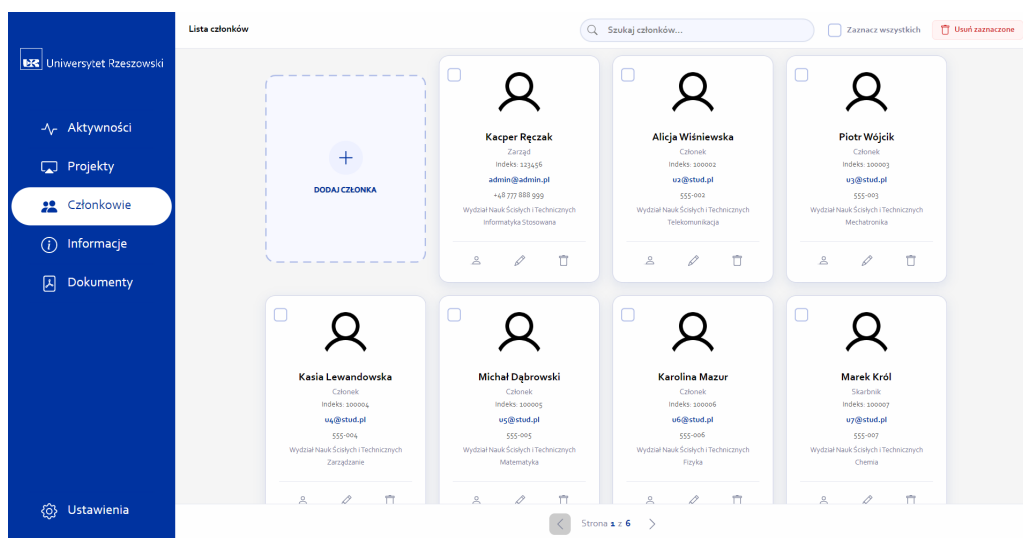
Zmiana aktywnego widoku w centralnym obszarze roboczym odbywa się w sposób w pełni dynamiczny – bez przeładowywania procesu całej aplikacji. Jest to realizowane przez mechanizm mapowania widoków na podstawie typu ViewModelu (*ViewLocator*). Każdy przycisk nawigacyjny w menu bocznym reaguje na najechanie kursorem (*hover effect*) płynną zmianą koloru tła oraz subtelnym przesunięciem ikony, a aktualnie aktywna zakładka jest wyraźnie oznaczona lewym marginesem w kolorze akcentowym systemu.

4.3 Moduł Członków Kół (Pełny cykl CRUD)

Moduł ten odpowiada za ewidencję i zarządzanie bazą członkowską organizacji naukowej. Wszystkie dane są przechowywane w bazie SQLite i podlegają rygorystycznym regułom walidacyjnym na poziomie modeli widoku przed jakąkolwiek próbą trwałego zapisu.

4.3.1 Odczyt danych (Read)

Lista członków wyświetla dane personalne studentów z podziałem na strony (paginacja po 9 elementów na stronę) w celu zachowania wysokiej wydajności interfejsu graficznego przy dużych zbiorach danych. Paginacja jest zaimplementowana w *MembersViewModel* i korzysta z zapytań LINQ z klauzulami *Skip* i *Take* na poziomie bazy danych. Na kartach profilowych prezentowany jest pełny profil studenta, zawierający m.in. nowo dodany numer telefonu kontaktowego, adres e-mail, kierunek studiów, rok akademicki oraz przypisaną rolę organizacyjną w kole naukowym (np. Prezes, Członek, Skarbnik).



Rysunek 4: Widok spisu członków koła naukowego z paginacją

4.3.2 Tworzenie rekordu (Create)

Proces dodawania nowego członka inicjowany jest kliknięciem kafelka "DODAJ CZŁONKA" o strukturze przerywanej linii (Dashed Stroke). Powoduje to otwarcie wysuwanego panelu modalnego po prawej stronie ekranu.

Formularz ten wymaga podania szczegółowych danych kandydata. Wpisanie danych wyzwała zdarzenie zmiany właściwości (**PropertyChanged**), które automatycznie uruchamia metodę **ValidateAddForm()**. Walidacja sprawdza poprawność formatu adresu e-mail za pomocą wyrażeń regularnych, unikalność adresu e-mail, a także poprawność numeru indeksu (który musi składać się wyłącznie z cyfr i mieć długość minimum 4 znaków). Przycisk "Dodaj członka" pozostaje nieaktywny (**IsEnabled=False**) dopóki wszystkie kryteria walidacji nie zostaną w pełni spełnione, a w przypadku błędów na dole formularza wyświetlany jest czytelny czerwony komunikat błędu.

Rysunek 5: Pusty formularz dodawania nowego członka koła

Dodaj nowego członka
Wypełnij dane nowego członka organizacji

Imię: Anna Nazwisko: Kowalska

Adres e-mail: anna.kowalska@ur.edu.pl

Numer telefonu: +48 500 600 700

Numer indeksu: 535999

Kierunek studiów: Informatyka

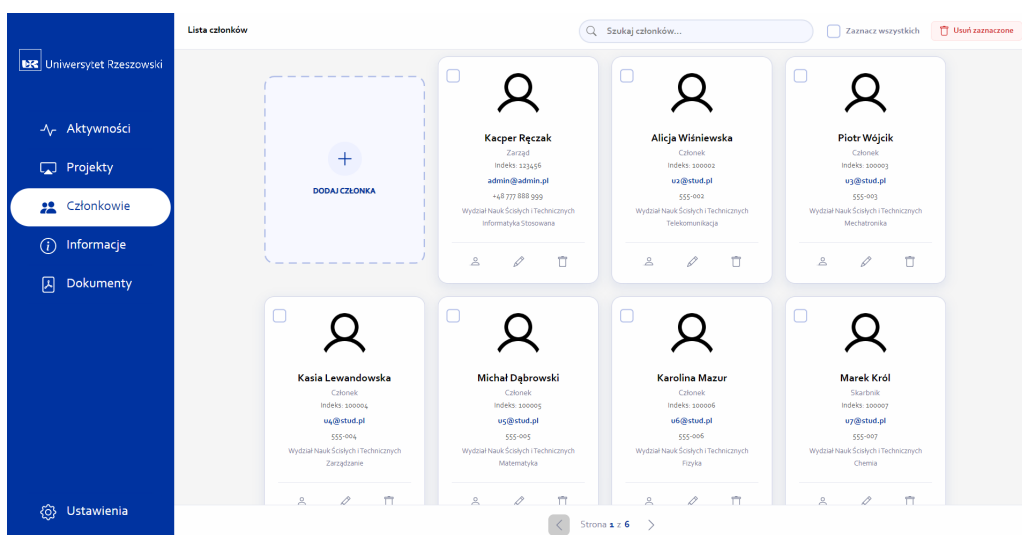
Wydział: Wydział Nauk Ścisłych i Technicznych

Rola w kole: Skarbnik Zdjęcie profilowe: Wybierz plik

Opis: Aktywny członek zarządu

Anuluj Dodaj członka

Rysunek 6: Formularz dodawania nowego członka wypełniony poprawnymi danymi



Rysunek 7: Spis członków po pomyślnym dodaniu nowego rekordu do bazy danych

4.3.3 Modyfikacja rekordu (Update)

Edycja danych członka pozwala na modyfikację numeru telefonu oraz pozostałych danych profilu. Po kliknięciu przycisku z ikoną ołówka na karcie członka, system ładuje aktualny model danych z bazy danych i otwiera popup modalny.

Podczas edycji system pilnuje, aby zmiana numeru telefonu lub innych pól nie powodowała naruszenia więzów unikalności w bazie danych SQLite (np. próba zmiany adresu e-mail na adres zajęty przez innego członka). Po zatwierdzeniu edycji przyciskiem "Zapisz zmiany" wywoływana jest metoda `UpdateMemberBasicInfoAsync` z poziomu repozytorium `MemberRepository`, która aktualizuje właściwości encji śledzonej przez `Entity Framework Core`, wywołuje transakcyjne zapisanie zmian w bazie (`SaveChangesAsync`)

oraz automatycznie unieważnia pamięć podręczną (`CacheService.Invalidate`), co natychmiastowo odświeża listę członków w głównym widoku aplikacji.

Rysunek 8: Formularz edycji profilu członka z załadowanymi aktualnymi danymi

Rysunek 9: Zmiana numeru telefonu członka w oknie edycji

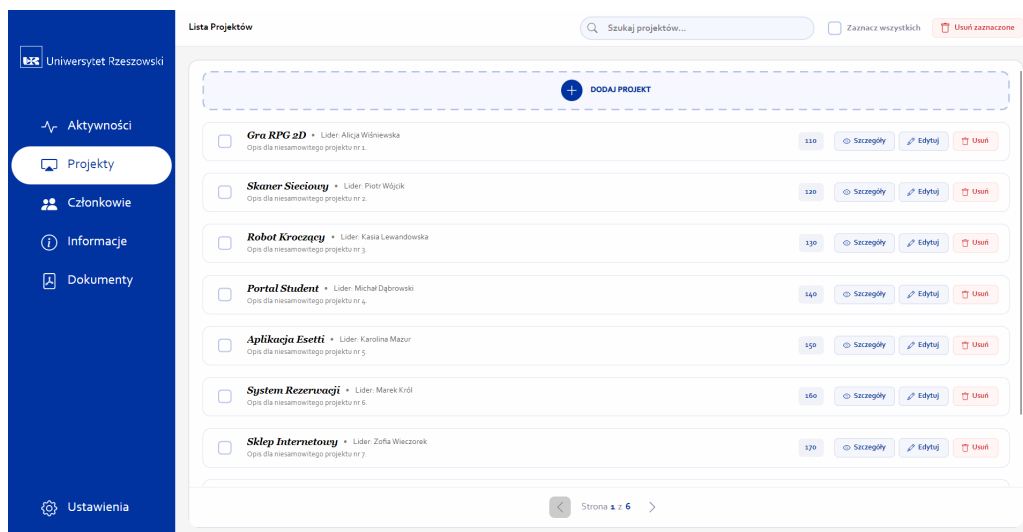
4.4 Ewidencja Projektów (Pełny cykl CRUD)

Moduł ewidencji projektów ułatwia koordynację prac badawczych, publikacyjnych i wdrożeniowych realizowanych przez członków koła naukowego.

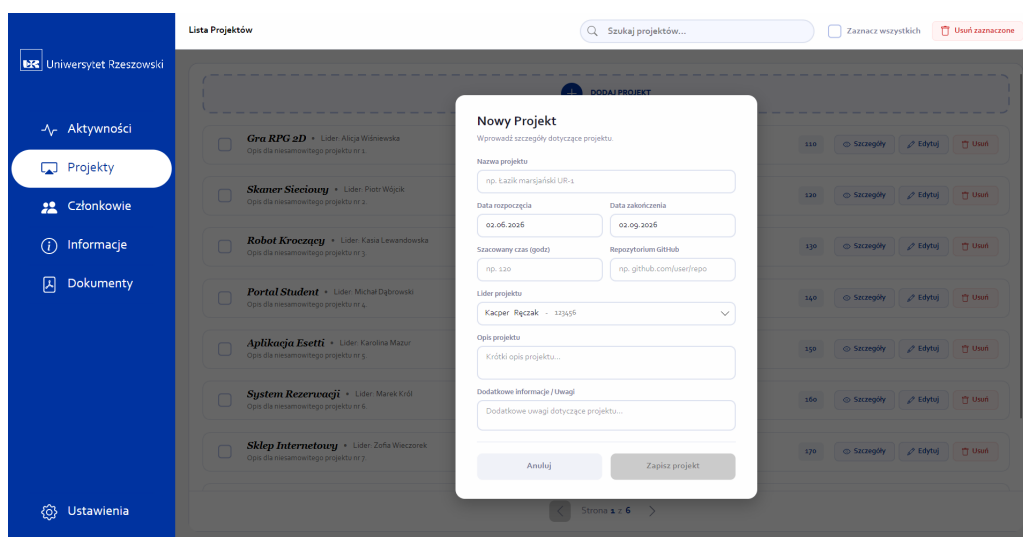
4.4.1 Odczyt danych i tworzenie (Read & Create)

Karta projektów prezentuje listę wszystkich projektów naukowych w postaci eleganckich kafelków. Każdy kafelek zawiera nazwę projektu, okres jego trwania, link do zdalnego repozytorium GitHub oraz imię i nazwisko kierownika projektu (lidera).

Dodanie nowego projektu odbywa się w popupie modalnym, gdzie użytkownik definiuje czas trwania, pracochłonność (w roboczogodzinach), oraz przypisuje członka koła jako lidera za pomocą rozwijanej listy (ComboBox) pobierającej dane bezpośrednio ze zmapowanej relacji bazy danych. Formularz kontroluje poprawność wprowadzanych dat (np. data zakończenia projektu nie może być wcześniejsza niż data rozpoczęcia) oraz wymaga podania nazwy o długości minimum 3 znaków.



Rysunek 10: Lista zarejestrowanych projektów naukowych



Rysunek 11: Pusty formularz rejestracji nowego projektu

Nowy Projekt
Wprowadź szczegóły dotyczące projektu.

Nazwa projektu:

Data rozpoczęcia: Data zakończenia:

Szacowany czas (godz): Repozytorium GitHub:

Lider projektu:

Opis projektu:

Dodatkowe informacje / uwagi:

Rysunek 12: Formularz dodawania projektu wypełniony poprawnymi danymi

Projekt	Lider	ID	Akcje
Gra RPG 2D • Lider: Alicja Wiśniewska Opis dla niesamowitego projektu nr 1.		110	Szczegóły Edytuj Usuń
Skaner Sיעיוני • Lider: Piotr Wójcik Opis dla niesamowitego projektu nr 2.		120	Szczegóły Edytuj Usuń
Robot Kroczący • Lider: Kasia Lewandowska Opis dla niesamowitego projektu nr 3.		130	Szczegóły Edytuj Usuń
Portal Student • Lider: Michał Dąbrowski Opis dla niesamowitego projektu nr 4.		140	Szczegóły Edytuj Usuń
Aplikacja Esetti • Lider: Karolina Mazur Opis dla niesamowitego projektu nr 5.		150	Szczegóły Edytuj Usuń
System Rezerwacji • Lider: Marek Król Opis dla niesamowitego projektu nr 6.		160	Szczegóły Edytuj Usuń
Sklep Internetowy • Lider: Zofia Wierczok Opis dla niesamowitego projektu nr 7.		170	Szczegóły Edytuj Usuń

Rysunek 13: Spis projektów zaktualizowany o nowo zarejestrowany projekt

4.5 Rejestr Aktywności i Spotkań (Pełny cykl CRUD)

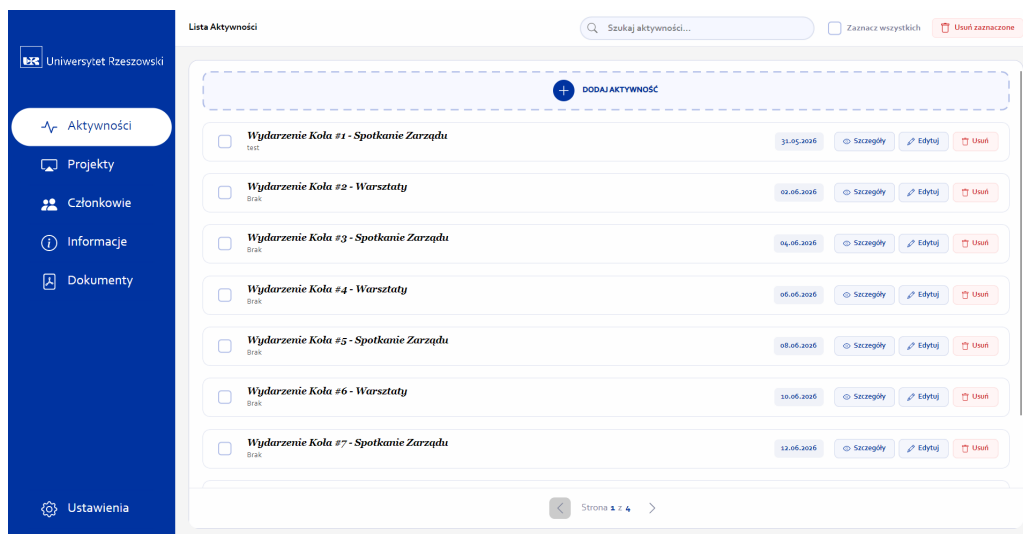
Służy do planowania oraz archiwizowania wydarzeń, seminariów naukowych, konferencji i cyklicznych spotkań organizacyjnych koła naukowego.

4.5.1 Odczyt danych i rejestracja (Read & Create)

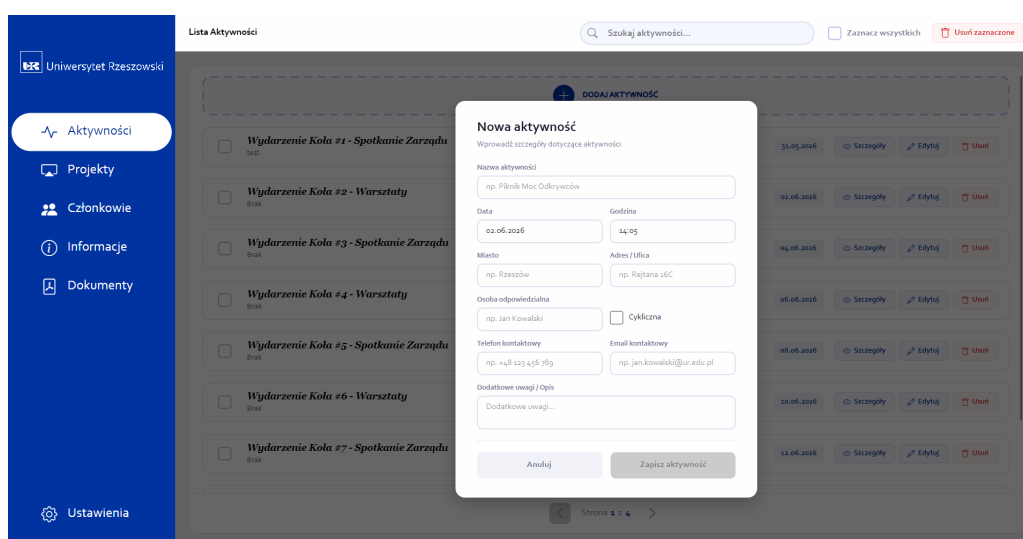
Każde wydarzenie w systemie Esetti posiada szczegółowo opisane miejsce (budynek, miasto, sala), termin, godzinę rozpoczęcia oraz dane osoby koordynującej spotkanie (imię, nazwisko, telefon, e-mail).

Podczas dodawania nowej aktywności system waliduje format numeru telefonu oraz poprawność adresu e-mail koordynatora za pomocą mechanizmu 'Regex'. Pomyślny zapis

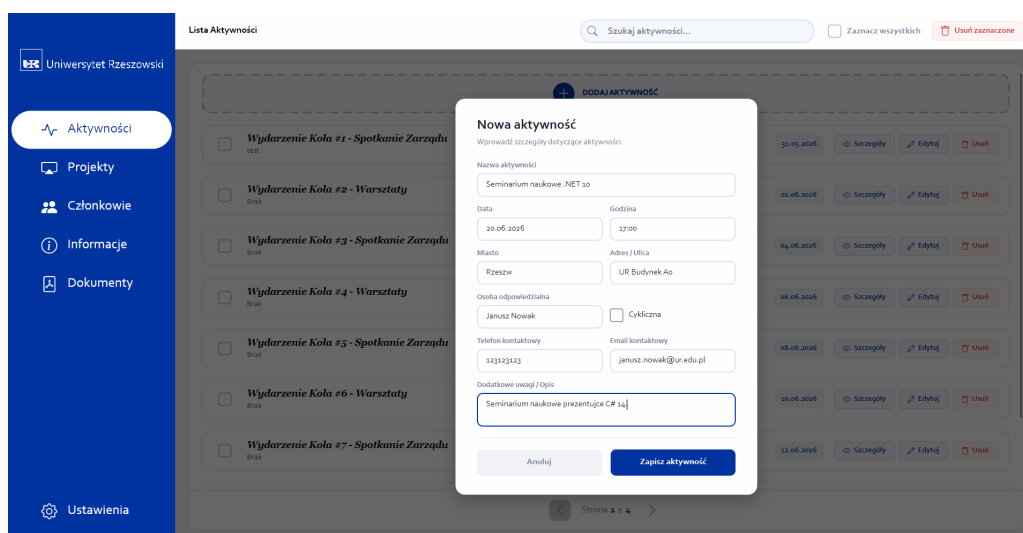
dodaje nowe wydarzenie na górę listy aktywności. Pozycja ta staje się również natychmiast widoczna na ekranie startowym (Dashboard) w sekcji nadchodzących wydarzeń.



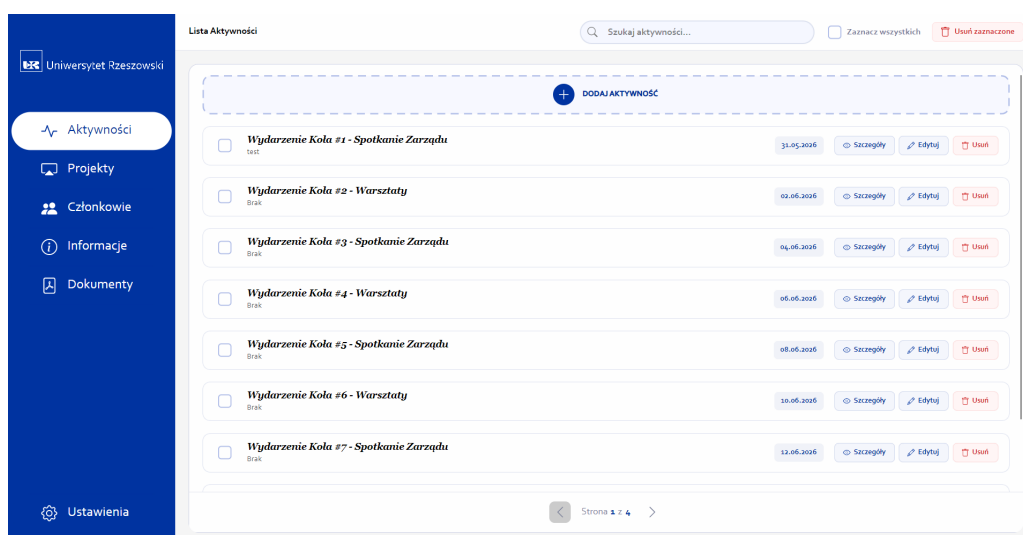
Rysunek 14: Panel aktywności i wydarzeń koła naukowego



Rysunek 15: Pusty formularz dodawania nowego wydarzenia



Rysunek 16: Kompletne informacje o nowym seminarium naukowym

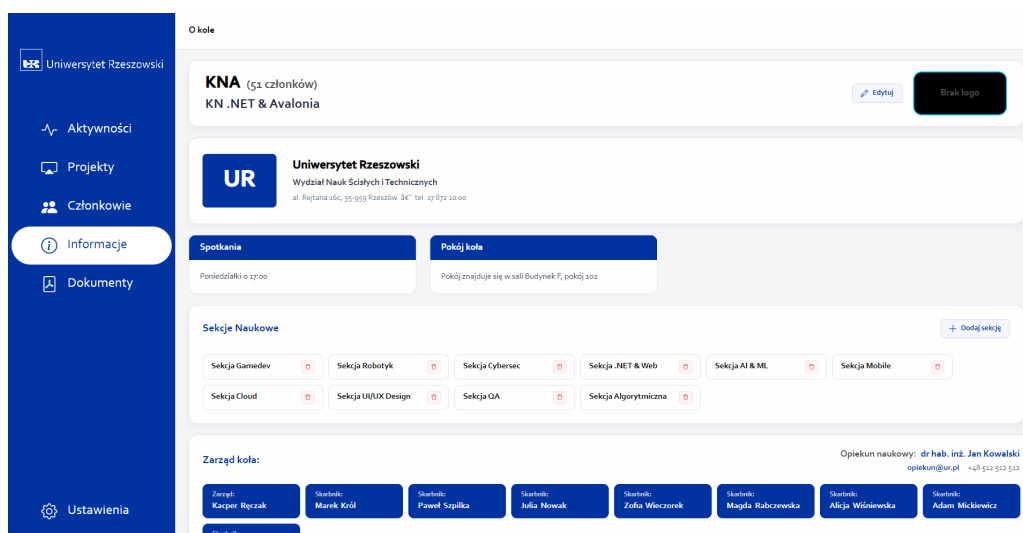


Rysunek 17: Lista aktywności zawierająca nowo dodane seminarium naukowe

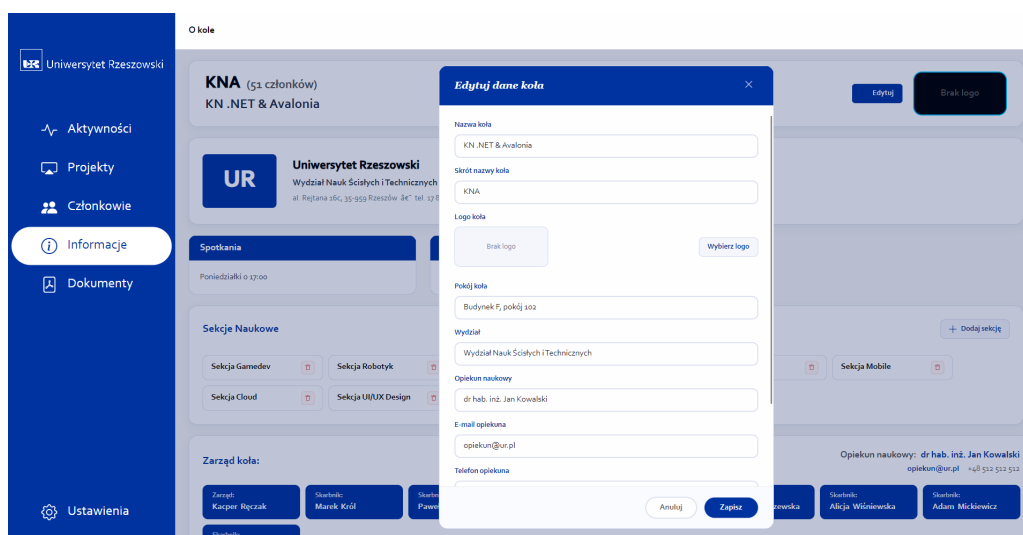
4.6 Informacje o Kole Naukowym (Opiekun naukowy)

Karta ta prezentuje formalne dane administracyjne koła naukowego. Wyświetla przynależność do uczelni, wydział, pokój koła, a także szczegółowy harmonogram stałych spotkań.

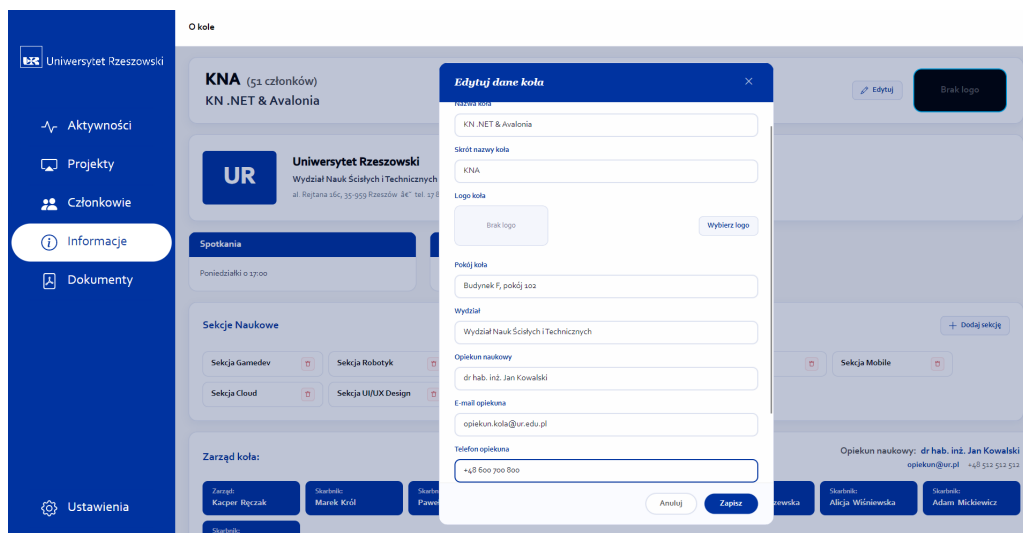
Kluczową funkcjonalnością jest prezentacja danych opiekuna naukowego koła – w tym jego imienia, nazwiska oraz wprowadzonych w tej iteracji bezpośrednich danych kontaktowych (adresu e-mail i telefonu). Popup edycji koła umożliwia modyfikację tych pól, zmianę logotypu koła (wczytywanego jako tablica bajtów 'byte[]' z pliku graficznego za pomocą okna dialogowego systemu operacyjnego) oraz edycję harmonogramu spotkań. Wewnętrzny ScrollView pozwala na wygodną nawigację wewnątrz popupu edycji na ekranach o niższej rozdzielczości, zapewniając pełną responsywność interfejsu użytkownika.



Rysunek 18: Widok informacji organizacyjnych o kole naukowym



Rysunek 19: Formularz edycji danych ogólnych koła



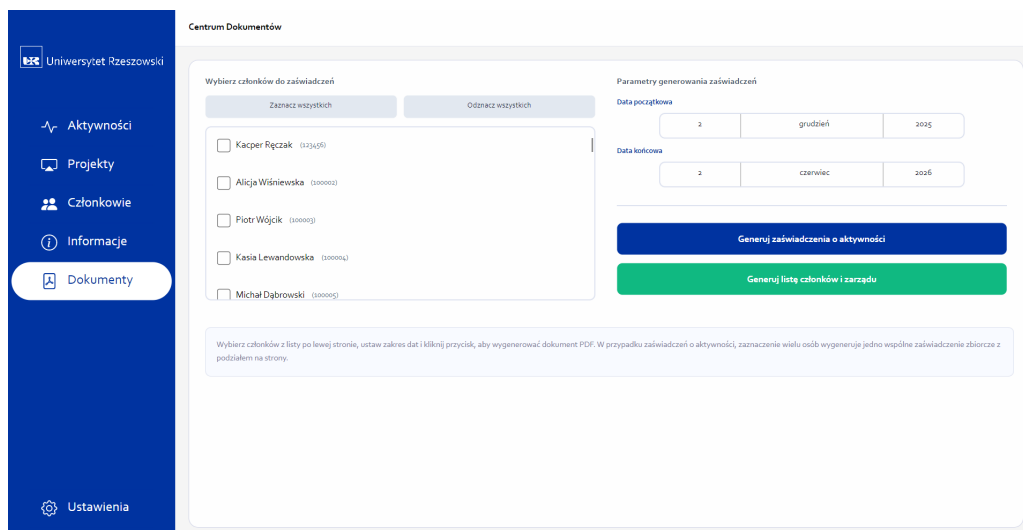
Rysunek 20: Wprowadzanie nowego adresu e-mail oraz telefonu opiekuna naukowego

4.7 Generowanie Dokumentów i Raportów

Moduł integruje zaawansowany silnik QuestPDF, który buduje dokumenty w oparciu o płynne API (Fluent API). Silnik ten nie konwertuje pośrednio kodu HTML/CSS, dzięki czemu generowanie plików PDF przebiega błyskawicznie i zużywa znikome ilości pamięci RAM.

Z poziomu interfejsu użytkownik może wygenerować dwa rodzaje raportów:

1. **Lista członków koła:** Generuje kompletne zestawienie wszystkich aktywnych studentów należących do koła naukowego, zawierające m.in. ich imiona, nazwiska, wydziały, role oraz nowo zaimplementowane numery telefonów. Dokument zawiera nagłówek z metadanymi uczelni, logotypem koła, klauzulą prawną dotyczącą ochrony danych osobowych (RODO) oraz miejscem na podpisy prezesa zarządu oraz opiekuna naukowego.
2. **Zaświadczenie o działalności:** Indywidualny dokument potwierdzający przynależność wybranego studenta do koła naukowego, zawierający historię jego zaangażowania w realizowane projekty oraz koordynowane aktywności w wybranym roku akademickim.



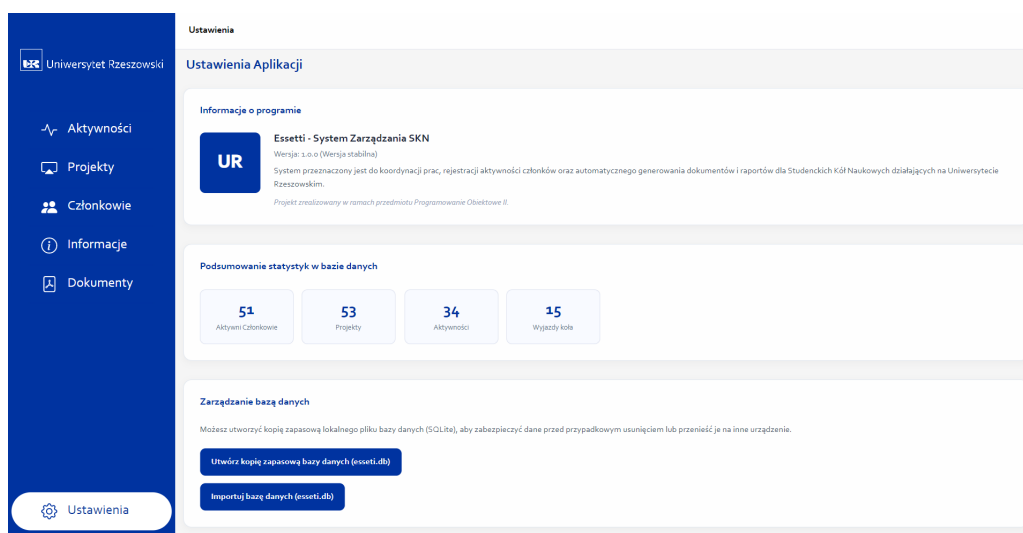
Rysunek 21: Panel wyboru i generowania dokumentów PDF

4.8 Panel Ustawień i Zarządzania Bazą Danych (Kopia zapasowa i Import)

Moduł ten udostępnia narzędzia do zarządzania plikiem SQLite oraz prezentuje globalne statystyki bazy danych (np. całkowitą liczbę członków, aktywności, projektów i delegacji wyjazdowych).

W pliku `SettingsViewModel` zaimplementowano proces importu bazy danych, który rozwiązuje problem blokowania plików przez Entity Framework Core podczas działania aplikacji:

- **Krok 1 (Zapis tymczasowy):** Zaimportowany przez użytkownika plik bazy danych nie nadpisuje od razu działającej bazy `esseti.db`. Jest on zapisywany pod nazwą `esseti_import.db`.
- **Krok 2 (Wyświetlenie komunikatu):** Aplikacja informuje użytkownika o konieczności ponownego uruchomienia aplikacji w celu sfinalizowania importu.
- **Krok 3 (Podmiana bazy przy starcie):** W pliku `App.axaml.cs` podczas startu wywoływana jest metoda `CheckAndSwapDatabase()`. Jeśli plik `esseti_import.db` istnieje, system zamyka połączenie EF Core, fizycznie podmienia plik `esseti.db` na nowy plik i czyści pliki dziennika zapisu SQLite (`esseti.db-wal` oraz `esseti.db-shm`), co całkowicie eliminuje błędy uszkodzenia struktury bazy danych.



Rysunek 22: Ekran konfiguracji, statystyk bazy i kopii zapasowych

5 Instrukcja Uruchomienia Aplikacji i Ograniczenia

Aplikacja Essetti dystrybuowana jest jako gotowy, przenośny pakiet wykonywalny dla systemu operacyjnego Windows, co minimalizuje trudności związane z instalacją środowiska produkcyjnego.

5.1 Wymagania Systemowe

Przed pierwszym uruchomieniem należy upewnić się, że system operacyjny spełnia poniższe wymagania:

- **System operacyjny:** Windows 10 (wersja 1809 lub nowsza) lub Windows 11.
- **Środowisko uruchomieniowe:** Zainstalowany pakiet redystrybucyjny .NET Desktop Runtime 9.0 (wersja x64).
- **Wolne miejsce na dysku:** Minimum 100 MB na pliki bazy danych i logotypy kół.
- **Uprawnienia:** Prawa do odczytu i zapisu w katalogu uruchomieniowym aplikacji (niezbędne do wykreowania pliku bazy SQLite).

5.2 Pierwsze Uruchomienie

W celu zainicjowania działania systemu należy postępować zgodnie z poniższymi krokami:

1. Pobrać spakowane archiwum ZIP zawierające skompilowaną aplikację.

2. Rozpakować całą zawartość archiwum do wybranego folderu lokalnego na dysku (zaleca się unikania folderów chronionych przez system, np. `Program Files`, ze względu na wymóg zapisu bazy danych).
3. Kliknąć dwukrotnie na plik wykonywalny `Esseti.exe`.
4. Podczas pierwszego uruchomienia system wykryje brak pliku bazy danych i automatycznie utworzy podkatalog `/Data` oraz zainicjuje plik `esseti.db`.
5. Zostaną automatycznie zaaplikowane migracje Entity Framework Core oraz uruchomiony seeder bazy danych, uzupełniający tabele o testowe dane startowe (w tym przykładowych członków, projekty oraz wydziały).

5.3 Znane Ograniczenia Systemowe

W obecnej wersji aplikacji występują następujące ograniczenia funkcjonalne i techniczne:

- **Lokalna baza danych:** Silnik SQLite nie wspiera współbieżnego dostępu z wielu niezależnych maszyn w sieci. Wszystkie edycje muszą być dokonywane lokalnie na danej instancji bazy.
- **Restart podczas importu:** Z powodu specyfiki działania biblioteki Entity Framework Core, która blokuje plik bazy danych podczas aktywności aplikacji, sfinalizowanie importu zewnętrznej bazy wymaga ręcznego ponownego uruchomienia programu.
- **Wielkość plików logo:** Pliki graficzne logo kół naukowych są zapisywane jako typ 'blob' bezpośrednio w bazie danych. Wrzucanie bardzo dużych zdjęć (powyżej 5 MB) może powodować wzrost rozmiaru bazy danych i spadek wydajności wczytywania szczegółów koła.

6 Weryfikacja Systemu (Instrukcja Testowa)

Poniższa sekcja zawiera procedury testowe służące do weryfikacji poprawności działania kluczowych modułów aplikacji Esetti.

6.1 Procedura Testowa: Cykl CRUD Członka Koła

W celu zweryfikowania poprawności dodawania, edycji oraz zapisu studentów, należy wykonać następujące kroki:

1. Przejdź do zakładki **Członkowie** za pomocą menu bocznego.

2. Kliknij kafelek **DODAJ CZŁONKA** i upewnij się, że przycisk potwierdzenia jest zablokowany.
3. Wypełnij pola poprawnymi danymi (np. Jan, Nowak, `jan.nowak@ur.edu.pl`, telefon: 123456789, indeks: 123456). Upewnij się, że przycisk odblokowuje się po poprawnym wpisaniu danych.
4. Kliknij **Dodaj członka** i zweryfikuj, czy karta nowego członka pojawiła się na końcu listy.
5. Kliknij ikonę **Edytuj** (ołówka) na nowo utworzonej karcie członka.
6. Zmień numer telefonu w polu formularza i kliknij **Zapisz zmiany**.
7. Sprawdź, czy numer telefonu zaktualizował się na kafelku profilowym.
8. Kliknij ikonę **Usuń** (kosza) i potwierdź operację. Karta członka powinna zniknąć z listy.

6.2 Procedura Testowa: Generowanie Dokumentów PDF

W celu potwierdzenia bezbłędnej pracy silnika QuestPDF należy przejść przez poniższe kroki:

1. Przejdź do zakładki **Dokumenty**.
2. Kliknij przycisk **Generuj listę członków**.
3. Wybierz lokalizację na dysku do zapisu pliku i kliknij **Zapisz**.
4. Otwórz wygenerowany plik PDF i sprawdź, czy zawiera on kompletny wykaz członków, w tym ich przypisane numery telefonów oraz stopkę z polami na podpisy władz koła.